MESA SCHOOL OF AI · FINAL EVALUATION · VIVA GUIDE

# VC Fundraising Pipeline Intelligence

Everything you need to explain the workflow, the components, the APIs, the backend and the frontend, with the reasons behind every choice and ready answers to the questions you are likely to get.

**Live site:** https://vc-pipeline-intelligence.vercel.app

**GitHub:** https://github.com/sisonpiy-droid/vc-pipeline-intelligence

**Demo video:** https://vc-pipeline-intelligence.vercel.app/video

**Components map:** in this guide (section 4 and the appendix) and in the repo at `docs/components-map.png`

**Viva:** 15 October 2026, 2 to 8 PM

### Check these before you walk in (5 minutes)

- **First real cron today.** The scheduled refresh is due at **09:30 IST** (it can start up to about an hour late). The 12 Oct run searched nothing because all 82 investors had been checked in the last 36 hours, so today is the first run that actually searches. Open `/runs` and look at "Latest run".
- **Telegram.** A daily report should have arrived around 11:00 IST on 13, 14 and 15 Oct. If you got them, say so. If not, say "connected and tested with a live message; the scheduled send is the part I am still confirming". Do not claim more.
- **Health.** Open `/api/health`: you want `"ok": true`. Keep the tab open as proof.
- **Demo tabs ready:** Home, a top investor page, Weekly brief, Runs & audit, API reference (Integrations panel), VC news, Investor database, the Components Map.

# 1. Your 60-second answer

> **"Vikram is a first-time founder raising a Seed round for a B2B logistics SaaS. He has 50 VCs and 32 angels to chase and no time to watch the news for all of them.** My system checks the public web for each investor's activity in the last 30 days, every Monday and Thursday. Gemini Flash reads the evidence and judges how relevant it is. **Fixed code, not the AI, turns that into a 0 to 100 follow-up score** and one recommended action. Only four columns are written back to Supabase, so his notes and contact history can never be touched. He opens the app and sees who to email first, why, with the source link and the exact quote that proves it."

### The five sentences to remember

1. **Evidence first:** no source link and verbatim quote, no recommendation.
2. **AI judges, code decides:** Gemini gives relevance (0 to 100); the score and action are arithmetic in code.
3. **A refresh can only write four columns:** enforced three ways (code schema, database role, tests).
4. **A failed search is never called "quiet":** unknown and quiet are different states, shown differently.
5. **A run is data, not a request:** a durable queue survives serverless time limits and crashes.

### Numbers to know cold

**82**
investors tracked: 50 VC firms + 32 angels

**30 / 30 / 20 / 15 / 5**
recency / relevance / evidence / fit / relationship

**75 and 55**
reach out now ≥ 75 · prepare outreach 55 to 74 · else research

**40**
relevance floor: below it a signal earns nothing

**30 days**
evidence window; undated pages are dropped

**4**
columns a refresh may write

**Mon + Thu 09:30 IST**
cron `0 4 * * 1,4` (UTC); may start ~1 h late on Hobby

**3 per claim · 240 s**
items per claim; time budget in a 300 s function

**900**
monthly search-credit budget; ~36 h skip rule

**15**
API routes (all documented in plain words)

**485 + 40**
unit tests + browser tests, all passing

**9**
database migrations

## 2. The brief, and how each requirement is met

**The problem (Problem Statement 10).** Vikram is raising a Seed round and tracks about 50 VCs in a Supabase table. Every few days he would have to search for news on each one by hand. He wants a system that does a twice-weekly signal refresh, tells him who to approach now, and never overwrites the notes and contact history he keeps in the same table.

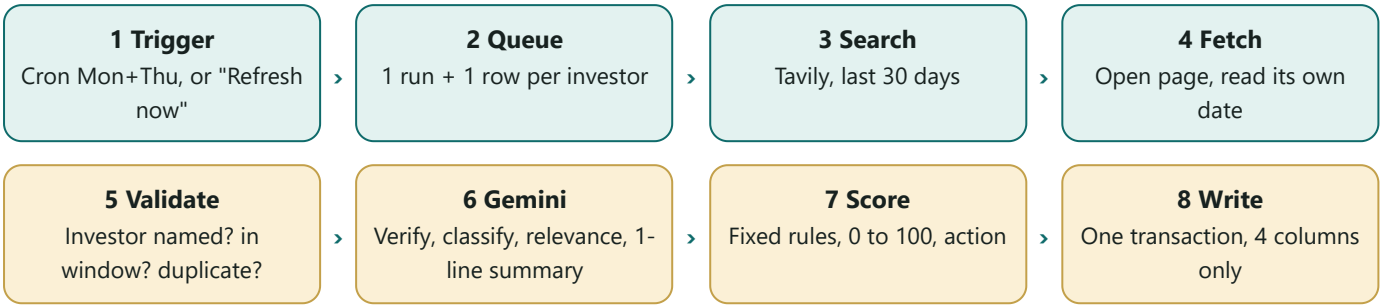
| REQUIREMENT | WHAT I BUILT | HOW IT IS PROVEN |
|---|---|---|
| Twice-weekly refresh of the investors | Vercel Cron every Monday and Thursday 09:30 IST starts a durable run over all 82 investors (the 50 VCs plus 32 angels from the workbook). | `vercel.json` + a test that keeps the dashboard's "next run" in step with it; `/runs` |
| Supabase table is updated | The `investors` table gets `last_signal_date`, `signal_summary`, `follow_up_score`, `recommended_action`. | Live database; four full production runs, 82/82, 0 errors |
| Never touch contact history or personal notes | Three layers: strict 4-field schema in code, a Postgres role that physically cannot read or write those columns, and tests that snapshot the row before and after. | `npm run verify:boundary` against the live database; tests |
| Live data, not a static list | Tavily search limited to the last 30 days with dated results only; each page is fetched and its own publication date is read. | Every signal shows its source link, date and retrieval time |
| Gemini Flash only | One model family. Claude is only the development tool, never called at runtime. A backup route reaches the same Gemini Flash model if Google is down (see Q&A). | `src/lib/ai/*`; the Integrations panel on `/api-docs` |
| Reasoning the founder can trust | Every recommendation has "Why this score": the five component numbers, the lead signal, the source and a verbatim quote. | Investor page, `docs/SCORING.md` |
| Deliverables | Vercel URL, GitHub repo, Components Map, a two-minute demo video (also on the site at `/video`). | All four exist and are linked on the cover |

### What "the core problem" is, so you can defend scope

The core is: refresh, score, write four columns safely, show the result. Everything else (chat, news, startups, cold search, Telegram, mail drafts) was added **on top** without changing that core: none of it writes to the four columns or to the founder's data, and each can be removed without breaking the pipeline.

## 3. The workflow, step by step

Two things happen on a schedule: the **refresh** (collect, judge, score, store) and the **daily report**. The founder's own actions (reading, editing notes, sending mail from his own app) are separate and never mixed into the refresh.




1. **Trigger.** Vercel Cron calls `GET /api/cron/refresh` with a secret ( `CRON_SECRET` ). Anyone without the secret gets 401. A manual "Refresh now" button exists, limited to one per 10 minutes to protect the free search credits.
2. **Plan the run.** The planner decides which investors are worth searching: it skips anyone checked in the last 36 hours, searches quiet investors about weekly, and stops when the monthly credit budget (900) is reached. It creates a `runs` row and one `run_items` row per investor.
3. **Process in chunks.** The function claims 3 investors at a time ( `FOR UPDATE SKIP LOCKED` ) until its 240-second budget ends, then hands the rest to a fresh invocation with an authenticated POST. Crash-safe: stuck items are reclaimed after 5 minutes.
4. **Research.** One Tavily search per investor for the last 30 days (dated results only). Each result page is fetched safely (SSRF-guarded, robots.txt respected) and its publication date is read from the page itself (JSON-LD or Open Graph).
5. **Validate in code.** Does the page name the investor? Is the date inside the window and consistent? Is it a real event rather than a profile or directory page? Is it a duplicate of another event? Each rejection stores a reason.
6. **Gemini Flash judges.** One batched call per investor returns structured JSON: is the investor genuinely involved, what kind of signal, relevance 0 to 100 to a B2B logistics Seed company, confidence, a one-sentence summary and a quoted excerpt. The quote must appear **verbatim** in the page we fetched, otherwise the item is rejected.
7. **Score in code.** Pure function over the stored signals, the investor's profile and relationship status: recency 30 + relevance 30 + evidence 20 + fit 15 + relationship 5, with a relevance gate. The action (reach out now, prepare outreach, wait, research, no action) follows from the score.
8. **Write safely.** One database transaction per investor, as the restricted role `vc_refresh` : save evidence, coverage and an audit line, then update only the four columns. If anything fails, the transaction rolls back and the investor keeps its previous score.
9. **Finish honestly.** A run is "succeeded" only when every investor finished; "partial" if some failed. The dashboard works out each investor's state when it reads: active, older signals, quiet (every source answered), needs review (partial), failed.
10. **Deliver.** The web app shows the ranked queue and the weekly brief. A daily Telegram message (05:30 UTC = 11:00 IST) summarises who to email first. The founder opens a draft and sends it from his own mail app. The app never sends email.

### Why I chose this workflow

| CHOICE | REASON | WHAT I REJECTED |
|---|---|---|

| | | |
|---|---|---|
| Durable queue (a run is data) | 82 investors × (search + several fetches + AI call) is minutes of work; a serverless function can be killed at 300 s. Storing the run in the database makes it resumable and gives an audit trail. | One long request; it would time out and lose half-finished work |
| AI judges, code scores | Models are good at reading text and bad at arithmetic and consistency. Fixed rules make the score explainable, testable and repeatable. | Asking the model for the final score or for dates and links |
| Evidence verification (verbatim quote, date from page) | It stops hallucinated sources. If the quote is not on the page, it does not count. | Trusting the model's own claims |
| A restricted database role for the refresh | The brief's hardest requirement is "never touch notes". A rule enforced by the database cannot be broken by a coding mistake. | Relying on careful code alone |
| Mon + Thu schedule | Even 3/4-day spacing; Vercel Hobby allows at most a daily cron; keeps ~710 search credits a month inside the free 1,000. | Daily (too costly), weekly (too stale for a fundraise) |
| Own pipeline, no agent framework | The steps are fixed, so a plain pipeline is cheaper, faster and auditable. Every step logs what happened. | LangChain or an autonomous agent: harder to test, can drift |
| Free tiers only | Vikram is a bootstrapped first-time founder; cost guards are part of the design. | Paid crawlers (Firecrawl, SerpAPI) and extra services |

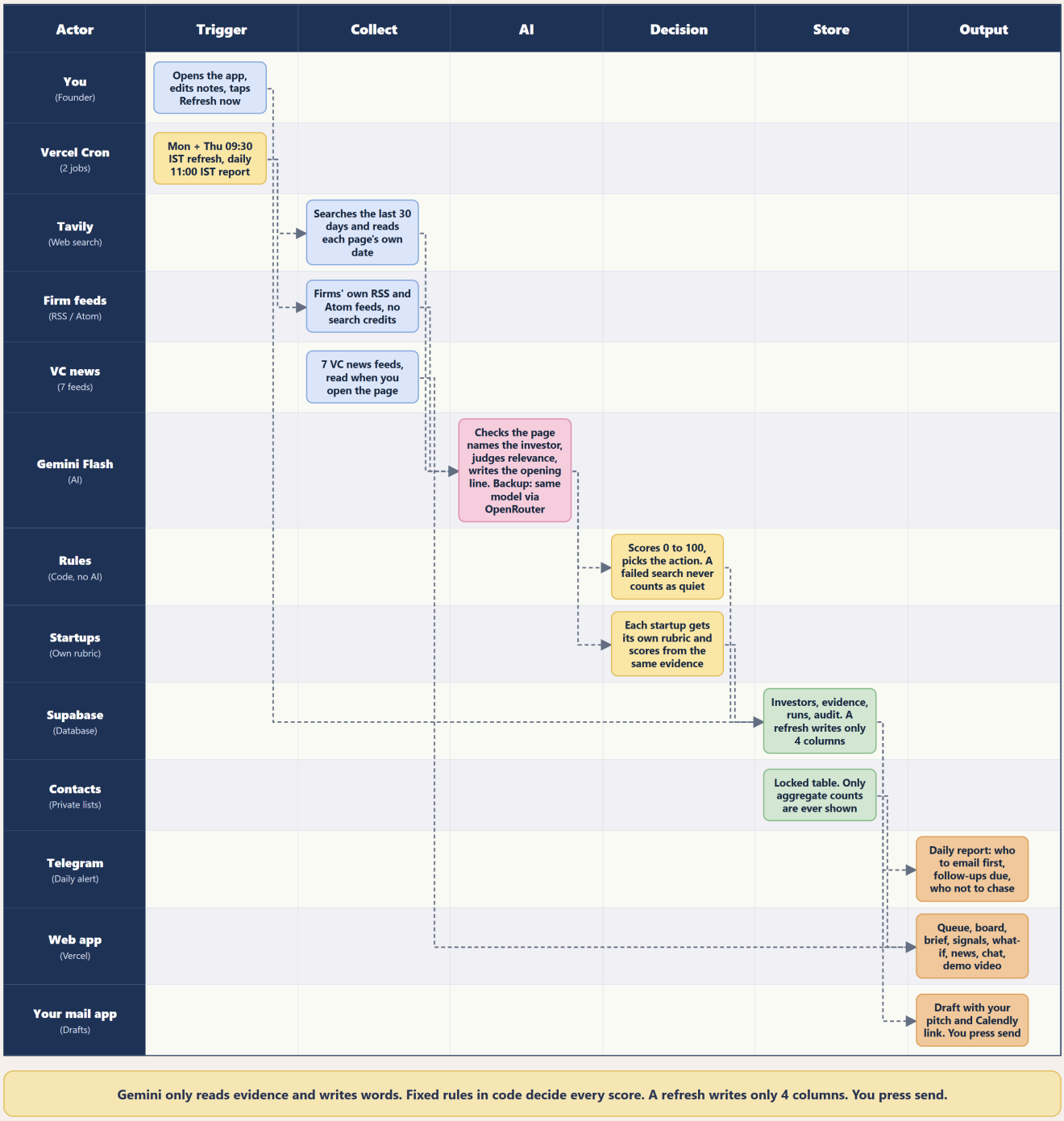
## 4. The Components Map (and why each part exists)

The map on the next page reads left to right: **Trigger → Collect → AI → Decision → Store → Output**. Each row is a real component. Boxes are coloured by role: blue (inputs), pink (AI), yellow (decisions made by rules), green (storage), orange (outputs). Dashed arrows are the data flow.

| COMPONENT | WHAT IT DOES | WHY THIS ONE |
|---|---|---|
| **Vercel Cron** | Starts the refresh Mon+Thu and the daily report | Built into the host, free, no extra scheduler to run |
| **Tavily** | Web search with date filters | Returns publication dates and can drop undated results, which a 30-day window needs |
| **Firm feeds (RSS/Atom)** | Reads firms' own feeds where they exist (9 of 50) | Free, dated, no search credits |
| **VC news (7 feeds)** | General industry news page, read on demand | Free public RSS; kept out of the scoring so it cannot affect scores |
| **Gemini Flash** | Judges evidence, writes opening lines, answers chat | Required by the brief; fast, cheap, supports structured JSON output |
| **Rules (code)** | Score, action, state | Deterministic and testable; no model in the arithmetic |
| **Startups engine** | Per-startup rubric and scores from the same evidence | Lets the tool serve a second startup without repeating searches |
| **Supabase (Postgres)** | Stores investors, evidence, runs, audit, startups | Required by the brief; real roles, grants and row-level security |
| **Contacts (private)** | Cold lists in a locked table; the app shows only counts and organisation-level search | Shows the data exists without exposing people's details |
| **Telegram bot** | Daily report to the founder's phone | Free, instant, the founder already uses it |
| **Web app (Next.js on Vercel)** | Queue, board, brief, signals, what-if, news, chat, video | One deployment serves both the pages and the API routes |
| **Mail drafts** | Opens a ready draft in the founder's own mail app, with his Calendly link | Keeps a human in the loop; the app never sends anything |

Use the map in the viva like this: start at the left (who triggers), follow one investor through the row of arrows, and end on the right (what Vikram sees).

# Components Map (swimlane)




Source file: `docs/components-map.png` (and `.html`) in the repository.

# 5. The APIs and services I used, and why

### External services

| SERVICE | WHAT FOR | WHY THIS ONE | LIMIT / IF IT FAILS |
|---|---|---|---|
| **Tavily Search** | Find recent pages about each investor | Date filters ( `start_date` , drops undated results); 1 credit per basic search | Free credits; guard stops spending at 900/month. A failed search keeps the old score and is flagged |
| **Gemini Flash** (Google) | Judge evidence, write opening line, chat | Required; structured JSON output; fast and cheap | Free-tier quota; bounded retries, then typed error; backup route (below) |
| **Supabase Postgres** | The database | Required; roles and row-level security; a direct connection with one secret | Free tier |
| **Vercel** | Hosting, serverless functions, cron | Push to main deploys; cron built in | Hobby plan: cron may start ~1 h late; 300 s function limit (hence the queue) |
| **Telegram Bot API** | Daily report | Free, simple HTTPS call | If not configured, the job quietly does nothing |
| **Public RSS feeds** | News page and firm feeds | Free, no key, published to be read by software | A failed feed is reported on the page, never hidden |
| **OpenRouter** (backup) | Same Gemini Flash model if Google fails | Keeps the product alive when the free quota runs out | Paid per use (cents per full run); only after Google has failed |
| **Calendly link, mailto/Gmail** | The meeting link and mail draft | No API needed: a URL and a mail-compose link | The app sends nothing |

**Deliberately not used:** Claude API at runtime (the brief says Gemini), Firecrawl or Jina (a page's date and text are enough), Crunchbase, SerpAPI, Brave (free-tier status unclear), MCP servers, any paid service. LinkedIn pages are never fetched (terms forbid it); only a search snippet can be used as weaker evidence.

### My own API routes (15), in plain words

| ROUTE | WHAT IT DOES | WHO CAN CALL IT |
|---|---|---|
| `GET /api/health` | Is the site alive and which settings are on (yes or no only) | Anyone |
| `GET /api/cron/refresh` | Scheduled twice-weekly refresh | Secret |
| `GET /api/cron/digest` | Daily Telegram report | Secret |
| `POST /api/refresh` | "Refresh now" button, one per 10 minutes | Passcode (off in open demo) |
| `POST /api/refresh/continue` | Internal hand-off for long runs | Secret |
| `POST /api/rescore` | Re-score from saved evidence; no search, no AI | Secret |

| | | |
|---|---|---|
| `POST /api/investors/[id]/founder` | Save the founder's notes, status, dates, history | Passcode (off in open demo) |
| `POST /api/demo/reset · /simulate` | Reset or simulate demo accounts only | Demo mode |
| `GET /api/palette` | Data for the Ctrl+K search | Anyone |
| `POST /api/chat` | Ask about the pipeline; answers from the dashboard's own data | Anyone, rate limited |
| `POST /api/startups (+ 3 more)` | Add, evaluate, edit the rubric of, and delete a startup | Passcode (off in open demo) |

Three kinds of access: **Public** (shows only what the dashboard already shows), **Passcode** (actions a person takes; switched off on the open demo site so examiners can try it), **Secret** (machines only; the secret never reaches a browser).

# 6. Backend deep dive

### The score (pure code, no model in the arithmetic)

| PART | POINTS | HOW IT IS WORKED OUT |
|---|---|---|
| Recency | up to 30 | Full points for a signal up to 3 days old, then falling in a straight line to 20% at day 30; multiplied by the relevance gate |
| Relevance | up to 30 | Gemini's 0 to 100 judgement of fit to "B2B logistics / supply-chain SaaS", scaled to 30 |
| Evidence | up to 20 | Confidence: high 1.0, medium 0.65, low 0.3; +0.15 if a second distinct relevant event exists; also gated |
| Fit | up to 15 | 40% sector, 40% stage, 20% geography (logistics and seed-stage India score highest; unknown scores lower) |
| Relationship | up to 5 | Not contacted 0.5, contacted 0.8, in conversation 1.0, passed or committed 0 |

**The two rules that matter more than the weights.** (1) **Relevance gate**: recency and evidence only count as far as the signal is relevant ( `relevance ÷ 70` , capped at 1). (2) **Relevance floor of 40**: below it a signal is stored but earns nothing. Without these, a fresh, well-sourced but irrelevant article would score **72**; with them it scores **18**.

#### Worked example A

2 days old, relevance 90, high confidence, seed-stage logistics investor in India, not yet contacted: 30 + 27 + 20 + 15 + 2.5 = **94.5** → **95**, "reach out now".

#### Worked example B

Same investor, but the article's relevance is 15: 0 + 0 + 0 + 15 + 2.5 = **18**. No signal credit; the score cannot exceed fit plus relationship (20).

**Actions (first match wins):** passed or committed → no action · awaiting reply (≤ 14 days) → wait · relevant and ≥ 75 → **reach out now** · relevant and 55 to 74 → prepare targeted outreach · relevant and < 55 → research further · no relevant signal but partial coverage → research further ("absence is not conclusive") · otherwise no action. The 75/55 thresholds come from a score grid in `docs/SCORING.md` ; the first version used 70/50 and let "adjacent" signals through.

### The write boundary (three independent layers)

1. **Code:** the only investor write is `applySignalUpdate` : a strict zod schema over exactly four fields; the SQL is built from a constant column list, never from the incoming object.
2. **Database:** the refresh connects as role `vc_refresh` : `UPDATE` on those four columns only, no `SELECT` on `personal_notes` or `contact_history` , so even a careless `select *` fails.
3. **Tests:** they snapshot the whole row before and after a refresh (success, repeats, partial failure, hostile model output) and assert it is identical outside the four columns, including `updated_at` .

### When things fail

| FAILURE | WHAT HAPPENS |
|---|---|
| Function killed mid-investor | Item reclaimed after 5 minutes; no half-written data (one transaction per investor) |
| Hand-off request lost | Heartbeat goes stale; the next trigger resumes the same run |

| | |
|---|---|
| Search or AI error | Retried with backoff and `Retry-After`, then up to 3 attempts, then the item fails and keeps its old score |
| Gemini quota or key problem | Second Google key, then the same model through OpenRouter if configured; otherwise a typed error and the run ends "partial" |
| Two triggers at once | A unique index allows one active run; the other gets "already running" |
| Button spam | 10-minute cooldown in the database |

### Security in one paragraph

Constant-time secret comparison and fail-closed cron auth (no secret set means nobody is allowed). Input validation with zod. A fetcher that blocks internal addresses (SSRF), checks the address it actually connected to, re-checks every redirect and obeys robots.txt. Passcode lockout and a database cooldown on the manual trigger. No secrets in the browser or in logs. Row-level security on with no public policies, so Supabase's automatic API exposes nothing. Web text sent to the model is treated as untrusted. Known gap, stated openly: the passcode lockout counter lives in each server instance's memory.

## 7. Database (Supabase Postgres)

| TABLE | HOLDS | OWNER OF THE DATA |
|---|---|---|
| `investors` | Profile, the founder's notes, status and history, and the 4 signal columns | Importer · **founder** · signal (refresh) |
| `signals` | One evidence row: URL, event date, retrieved-at, quote, summary, relevance, confidence, verdict; rejected ones too, with the reason | Refresh |
| `runs · run_items` | Run history and the per-investor queue with score breakdown | Refresh |
| `source_coverage` | Latest status for each investor and each source (ok, empty, error, skipped) | Refresh |
| `audit_logs · provider_calls · score_history` | Events (never notes), provider health, score over time | System |
| `portfolio_companies` | What each firm already backs, to flag possible competitors | Refresh |
| `startups (+ 3 tables)` | Per-startup rubric, workflow, scores and cached AI reviews | Founder |
| `investor_directory` | The private cold lists. Row-level security on, no grants; only aggregate and organisation-level queries read it | Private |

**Why Postgres directly instead of supabase-js:** fewer secrets (one connection string), access to roles and grants, and the same SQL runs in PGlite for the tests, so the safety tests exercise the real migrations.

## 8. Frontend

**Stack:** Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS 4, GSAP for motion, Phosphor icons, Geist fonts. Pages are server components that read Postgres directly, so there is no browser-side database client and no secret in the browser. The same deployment hosts the API routes.

| PAGE | WHAT IT SHOWS |
|---|---|
| **Priority queue** `/` | Greeting, counts that add up to all investors, "Your week in 30 seconds" cards, "Start here" picks with source and opening line, who not to chase |

| | |
|---|---|
| **Outreach board** `/board` | Kanban of relationship status; "Reach out now" opens the mail draft and moves the card to Contacted |
| **Weekly brief** `/brief` | The whole reach-out pool (strong, good, possible fits), new signals, score movers, follow-ups; printable |
| **Pipeline · Investor detail** | Search and filters; evidence timeline, score breakdown, founder edit form, mail draft, reply log |
| **Signals · Quiet investors** | Sortable evidence feed; quiet vs failed vs partial kept apart |
| **What-if ranking** | Re-rank with other weights using the same scoring function; nothing saved |
| **VC news** | Read-these-first, plain-words topics, "matters to you" flag, search |
| **Startups · Investor database** | Multi-startup workspace; cold search by organisation (no personal data) |
| **Runs & audit · API reference** | Run history, credits, evidence funnel · routes in plain words plus an Integrations panel |
| **Demo video** `/video` | Two-minute walkthrough with chapters |

### Design decisions you can defend

- **Answer first.** The home screen leads with "who to email this week", not a table of 82 rows.
- **Plain words.** Labels like "Couldn't check yet" instead of jargon, and a "How we decided" explanation.
- **Honest states.** Failed, partial and quiet look different, so the founder is never told "nothing happened" when a search failed.
- **Light and dark mode** that follows the device and can be switched; **responsive** down to phone width (every page is tested at phone width); keyboard focus, a skip link and labelled controls; reduced-motion respected.
- **GSAP** page choreography: titles rise in, rows and cards rise as they scroll into view; the page is fully visible without JavaScript.

## 9. Extras beyond the brief, and how I verified everything

### Added on top (core untouched)

- Chat assistant over the dashboard's own data
- VC industry news page
- Multi-startup rubrics
- Cold search over the private contact lists (organisations only)
- Email drafts with a Calendly link; the app never sends
- Daily Telegram report
- What-if ranking, outreach board, weekly brief
- Demo video page and API reference with an Integrations panel
- AI backup route to the same model

### Verification

- **485** unit and integration tests (Vitest, real migrations on PGlite)
- **40** browser tests (Playwright, production build, phone widths)
- Full production runs completed **82/82** investors with **0 errors**
- **10/10** golden AI cases on live Gemini
- Write boundary verified on the **live** database
- Typecheck and production build clean

**Be honest about these (examiners respect it)**

- Recall is the weak point: one Tavily query per investor and no independent crawl of every firm's site. "No news found" is only claimed when every source answered.
- The first scheduled run that actually searches is today's 09:30 IST cron. Earlier proof comes from the full production runs that completed 82/82 with no errors.
- The open demo has no password on purpose, so examiners can try it. A real deployment would add sign-in.
- Score weights are a reasoned starting point (from the brief); they should be recalibrated against real outcomes.

# 10. Likely questions and how to answer them

Say it in your own words. Each answer is written to be spoken in 20 to 40 seconds.

### Problem and design

**Q1. Explain your project in one minute.**
Use the 60-second answer on page 2. Finish with: "and the key promise is that the refresh can only write four columns, so the founder's notes are safe."

**Q2. Why did you choose this workflow?**
Because the work is slow and can fail partway: 82 investors, each needing a search, page fetches and an AI call. So I made a run into data in the database (a queue). That survives serverless time limits and crashes, gives an audit trail, and lets me resume. The steps themselves are fixed, so a plain pipeline is cheaper and easier to test than an agent.

**Q3. Where exactly is the AI used, and where is it not?**
AI reads and judges text: is the investor really involved, what kind of event, how relevant to logistics SaaS, a one-sentence summary, an opening line, and the chat. It does not choose dates, links or the score. Those come from page metadata, the search provider and code.

**Q4. The brief says about 50 VCs. Why 82?**
The workbook has 50 VC firms and 32 angels. I tracked all of them because angels matter at Seed. The demo accounts are labelled and never counted in the real numbers.

**Q5. What is the single most important design decision?**
Separating who owns what. The founder owns notes and contact history; the refresh owns four signal columns. I enforced that in the database with a restricted role, not just in code, because a rule the database enforces cannot be broken by a bug.

### AI and scoring

**Q6. How do you stop the AI hallucinating?**
Four ways. Dates come from the page's own metadata, not the model. Links come from the search provider. The model must return structured JSON that is validated. And its quoted evidence must appear word for word in the page I fetched, otherwise the item is thrown away.

**Q7. How is the score calculated?**
Recency 30, relevance 30, evidence 20, fit 15, relationship 5. Gemini only supplies the relevance number; code does everything else. Recency and evidence are gated by relevance, and anything under relevance 40 earns nothing, so a fresh irrelevant article cannot score high.

**Q8. Why those thresholds, 75 and 55?**

I built a grid of scores by age and relevance for a perfect-fit investor. 75 needs a fresh, clearly relevant signal; a merely adjacent one (relevance about 55) lands in "prepare outreach" instead. My first version used 70 and 50 and let adjacent signals through, so I raised them. They are constants and should be recalibrated with real outcomes.

HARD **Q9. Why did you change the brief's weights? (You did not.)**

I kept 30/30/20/15/5. What I added is structure: the relevance gate and the floor of 40. With the literal weights, a fresh, well-sourced, irrelevant article scored 72; with the gate it scores 18.

**Q10. What if the web page tries to trick the model (prompt injection)?**

Page text is treated as untrusted data, the model can only return a fixed JSON shape, it has no tools to call, and its quote must exist in the page. The chat fences web text as untrusted and answers only from the dashboard's own data.

**Q11. How did you test the AI part?**

Provider doubles for fast, repeatable tests (including hostile model output), plus 10 golden cases against live Gemini that all pass. The score itself is pure code, so it has ordinary unit tests.

HARD **Q12. The brief says Gemini Flash only. Why is there an OpenRouter backup?**

It is the same Gemini Flash model, reached through a different door, and it is used only if Google fails, for example when the free quota runs out. The product never switches to a different model. It is optional, logged, and shown in the Integrations panel; removing the key turns it off. If you would rather see strictly Google-only, I can remove the key and the behaviour reverts to a clear error.

## Backend and infrastructure

**Q13. How do you handle 82 investors with a 300-second limit?**

A run is rows in the database. Each invocation claims 3 investors at a time with `FOR UPDATE SKIP LOCKED` until 240 seconds, then hands off to a fresh invocation. If a function dies, items are reclaimed after 5 minutes; if the hand-off is lost, the next trigger resumes the same run.

**Q14. What does "quiet" mean, and how is it different from "failed"?**

Quiet means every source answered and found nothing relevant in 30 days. Failed or partial means a search did not complete, so the activity is unknown. A failed item writes nothing and keeps its previous score, and the screen says "couldn't check yet", never "quiet".

**Q15. How do you keep search costs inside the free tier?**

A planner skips investors checked in the last 36 hours, searches quiet investors about weekly, and enforces a monthly credit budget of 900. The Runs page shows the credits used so far.

**Q16. Why Monday and Thursday?**

Vercel's free plan allows at most one cron a day, with up to about an hour of jitter. Monday and Thursday give even three- and four-day gaps and keep about 710 searches a month under the free allowance. The schedule is in `vercel.json` and a test keeps the dashboard's "next run" in step.

**Q17. What happens if two refreshes start at once?**

A partial unique index in the database allows only one active run, so one wins and the other is told "already running". Overlap protection is the database's job, not application logic.

**Q18. Why Next.js for both the UI and the API?**

One deployment and one language. Server components read the database directly, so no database client or secret reaches the browser, and the API routes sit next to the pages that use them.

**Q19. Why Postgres directly and not the Supabase client library?**

One secret instead of several, direct use of roles and grants (which the write boundary needs), and the same SQL runs in PGlite in the tests. Row-level security is still on with no public policies, so Supabase's automatic API exposes nothing.

### Data safety and privacy

**Q20. How do you guarantee the notes are never overwritten?**

Three layers: a strict four-field schema in code, a database role that cannot read or write those columns, and tests that compare the whole row before and after. I also verified the role against the live database.

**Q21. Is the website secure? Anyone can open it.**

Yes, it is an open demo by choice so examiners can try it. Secrets stay on the server. Machine routes need a secret and fail closed. Manual refresh has a cooldown, and the chat is rate limited and cannot see notes or contacts. A real deployment would add sign-in; I say so in the docs.

HARD **Q22. You hold thousands of private contacts. Is that shown publicly?**

No. They sit in a locked table with row-level security and no grants. The app only counts them and offers a search that lists organisations, never a person's name, email, phone or profile. A test fails if any personal field ever appears in a result, and another checks that nothing in the app writes to that table.

**Q23. What is SSRF and how do you prevent it?**

It is tricking a server into fetching internal addresses. My fetcher blocks private and internal addresses, validates the address it actually connected to (so DNS tricks fail), re-checks each redirect, and respects robots.txt.

**Q24. Does the app send emails?**

Never. It prepares a draft with your pitch and Calendly link and opens it in your own mail app. Moving a card to Contacted is a founder action with one-click undo, because opening a draft is not proof it was sent.

### Frontend

**Q25. Why did you design the home screen this way?**

The founder wants an answer, not a spreadsheet. So it leads with the investors to email this week, each with its evidence, then the counts, then the rest. Everything else is one click away.

**Q26. What did you do for accessibility and responsiveness?**

A skip link, labelled controls, visible keyboard focus, readable contrast in both light and dark themes, reduced-motion support, and browser tests at phone widths that fail on horizontal scrolling or console errors.

**Q27. Where is GSAP used, and does the page work without it?**

For the entrance of titles, the cards, and rows that rise as you scroll. Content is only hidden by the script, never by the stylesheet, so the page is fully visible without JavaScript; with reduced motion nothing animates.

## Limits and the future

**Q28. What is the weakest part?**

Recall. One search per investor and no crawl of every firm's site, so I can miss news. I mitigate it by being honest: coverage gaps are shown and "no signal" is only claimed when every source answered. Next step: per-firm feeds or sitemaps with snapshot diffing.

**Q29. How would it scale to 5,000 investors?**

The queue already scales horizontally, since work is split into small claims. Cost grows linearly with searches and AI calls, so I would move to paid search and AI tiers, batch by priority, and search quiet investors less often.

**Q30. What would you build next?**

Reply tracking so the app learns from outcomes, recalibrating the weights from real results, deeper per-firm crawling, and sign-in with Supabase Auth for a real deployment.

HARD **Q31. What is not finished or not verified?**

Be direct: "The first scheduled run that actually searches is today at 09:30 IST; earlier runs prove the pipeline end to end. Telegram is connected and tested with a live message. The weights are reasoned, not yet calibrated on outcomes."

**Q32. Did you consider an MCP server or an agent framework?**

Yes, and left them out on purpose. The brief needs a scheduled job and a database, not an agent interface. An MCP server would be a second way into the data, which weakens the main guarantee. A fixed pipeline is easier to test and audit than an agent.

**Q33. If Gemini is down at 09:30 on a cron day, what does the founder see?**

The system tries the backup route. If that is also down, those investors fail, keep their previous scores, and are labelled "couldn't check yet". The run ends "partial", the Runs page shows why, and the next trigger retries. Nothing wrong is written.

**Q34. How do you know a score is right?**

Every score shows its five components, the lead signal, the source and the quote, so a person can check it in seconds. The thresholds were chosen from an inspectable grid, and the what-if page shows how changing weights moves the ranking.

> **Q35. What is the difference between "fit" and "relevance"?**
> Relevance is about the signal: how much a piece of news matters to a B2B logistics Seed company. Fit is about the investor: sector, stage and geography. Both are kept apart in the breakdown, along with relationship readiness and evidence confidence.

## 11. Live demo script (about 4 minutes)

1. **Home** `/` : "The answer is first." Point at the greeting, the "N of 82 investors have a fresh signal" headline (31 when I last checked), the story cards and "Start here". *(30 s)*
2. **Click the top pick.** Show the score breakdown, the exact quote and the source link. Say: "AI judged relevance; code did the maths." *(60 s)*
3. **Preview email.** Show the draft with the Calendly link. "The app never sends it." *(20 s)*
4. **Weekly brief** `/brief` : the whole reach-out pool. *(20 s)*
5. **Runs & audit** `/runs` : today's cron run, credits used, the evidence funnel and "failed is never quiet". *(40 s)*
6. **API reference** `/api-docs` : open the Integrations panel. "Names only, never values." *(20 s)*
7. **Components Map:** walk one investor left to right. *(40 s)*
8. **Bonus if asked:** chat, VC news, Investor database cold search (organisations only), demo video. *(30 s)*

### If something breaks

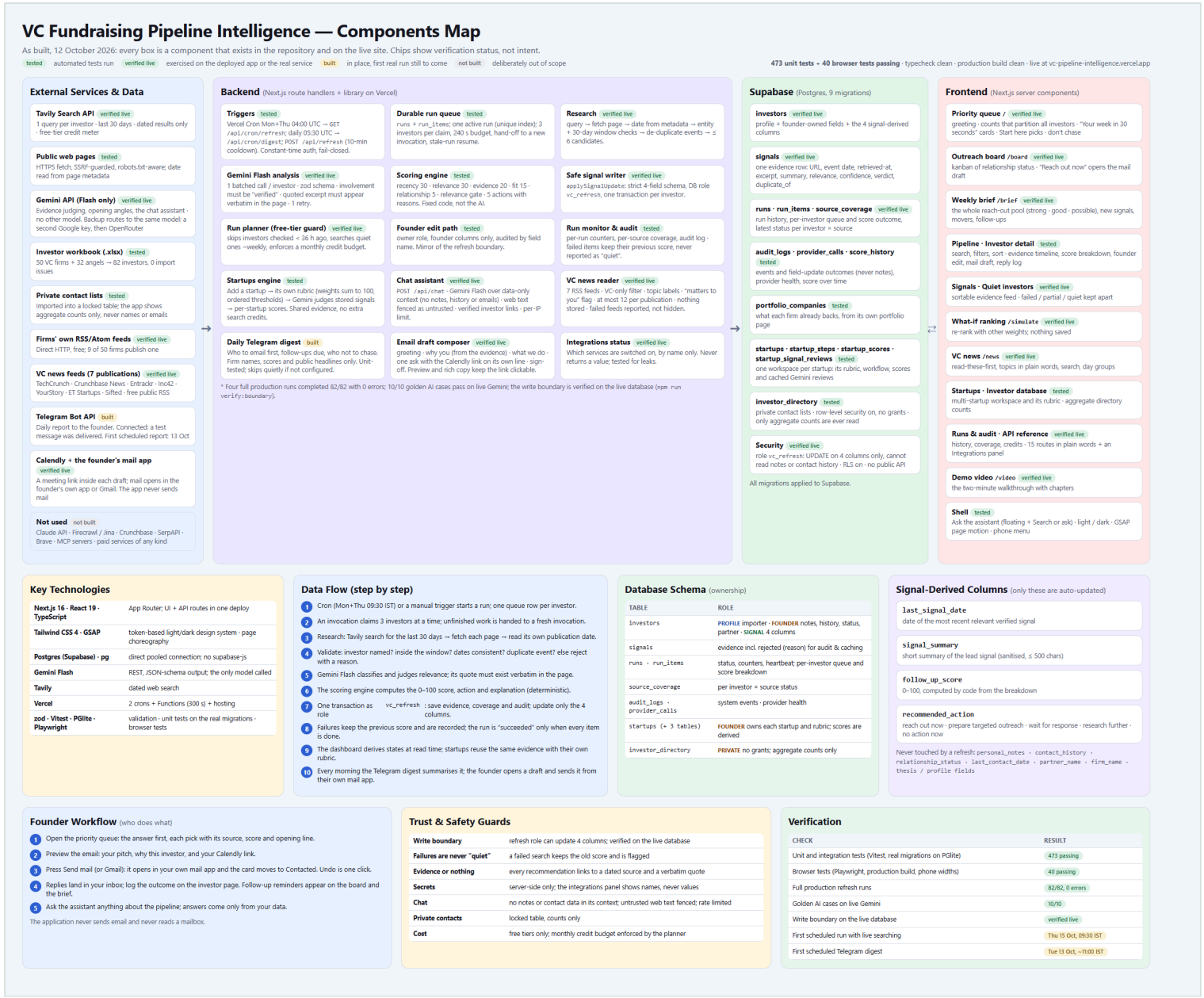
- **Page slow or blank:** reload once; Vercel serverless can cold-start. Then switch to the demo video at `/video` .
- **Chat does not answer:** say "it falls back to a plain summary when Gemini is unavailable" and show that fallback; do not retry repeatedly (rate limit).
- **Cron has not run:** show `/runs` and the schedule ("0 4 * * 1,4 UTC, up to an hour late on the free plan"), plus the earlier completed runs.
- **Examiner asks to see code:** `src/lib/refresh/*` (pipeline), `src/lib/scoring/score.ts` , `supabase/migrations/0001_init.sql` (roles), `tests/` .

## 12. Plain-words glossary

| TERM | MEANING |
|---|---|
| **Cron** | A timer that calls a URL on a schedule. Here: Monday and Thursday for the refresh, daily for the report. |
| **Serverless function** | Code that runs only when called and is stopped after a time limit (300 s here). That is why a run is split into chunks. |
| **Durable queue** | The to-do list stored in the database, so work survives crashes and restarts. |
| **Row-level security (RLS)** | A Postgres switch that denies access to rows unless a policy allows it. Here it is on with no public policy. |
| **Database role / grant** | A login identity and the exact rights it has. The refresh's role can update four columns and nothing else. |
| **zod** | A library that checks data has exactly the expected shape, used on AI output and API input. |
| **SSRF** | Tricking a server into fetching internal addresses; blocked by the safe fetcher. |

| | |
|---|---|
| **Idempotent** | Doing it twice has the same result as once. Re-running an import or a refresh does not create duplicates. |
| **Hallucination** | When a model states something untrue. Countered by verifying dates, links and quotes in code. |
| **Structured output** | Forcing the model to answer in a fixed JSON shape instead of free text. |
| **PGlite** | A tiny Postgres that runs inside the tests, so tests use the real migrations. |
| **Signal** | One dated, sourced piece of evidence about an investor (a fund close, a new investment, an interview). |

# Appendix: detailed Components Map (with verification chips)




Green chips mean tested or verified live; amber chips mean built but the first scheduled run is still to come. Source: `docs/components-map-detailed.png`.